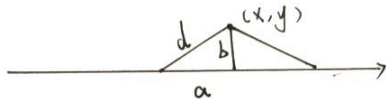


## 第六周 动态规划（下）与贪心算法（上） 作业

第一题：

1、~~设~~ 设  $(y_1, y_2, \dots, y_n)$  为一个最优解，则  $(y_2, y_3, \dots, y_n)$  是下面子问题的最优解。  
 假设  $(z_2, z_3, \dots, z_n)$  是最优解，即  $(y_2, y_3, \dots, y_n)$  不是最优解  $\sim \max \sum_{i=2}^n v_i x_i$ 。  
 则  $\sum_{i=2}^n v_i y_i \leq \sum_{i=2}^n v_i z_i$ 。  
 $\Rightarrow \sum_{i=1}^n v_i y_i \leq \sum_{i=1}^n v_i z_i + v_1 y_1$ ，说明  $(y_1, y_2, \dots)$  更优，假设矛盾。

第二题：



对于每个岛可以投影其在海岸线上的区间

转化为用最少的点，使得所有区间至少有一个点。

```
class Solution{
public:
    int radarInstallation(vector<Island>& islands, double d){
        int n=islands.size();
        double radar_x=0;
        sort(islands.begin(), islands.end(),cmp); //对岛屿进行从小到大的排序
        int res=1;
        if(islands[0].y>d){
            return -1;
        }
        else{
            radar_x=islands[0].x+sqrt(pow(d,2)-pow(islands[0].y,2));
        } //然后判断剩下的在不在雷达的范围内
        for(int i=1;i<islands.size();i++){
            //一次判断，如果不满足题意，就更新x坐标
            if(sqrt(pow(islands[i].x-radar_x,2)+pow(islands[i].y,2))>d){
                //超过这个范围就更新新的雷达坐标
                radar_x =islands[i].x+sqrt(pow(d,2)-pow(islands[i].y,2));
                res++;
            } //如果<d，就跳过这个
        }
        return res;
    }
};
```

第三题：

对于空间上的优化，可以将二维数组变为滚动数组，从而变成一维。从整个程序来看，我们可以将每一层的循环判定的存储矩阵分为两块， $dp[i-1][v]$ ， $dp[i-v][v-w[i]]$ 。也就是一部分是过去存储的最优dp值，一部分是 $dp[i-1][v]$ 。所以我们可以设立一个一维数组，然后从右往左遍历，直接覆盖 $dp[i-1][v]$ ，从而只需要一个固定数组就可以，从而空间只需 $O(C)$ 。

第四题：

一.  $dp[i][j]=\max(dp[i-1][j], dp[i-1][j-w[i]]+v[i], dp[i-1][j-2*w[i]]+2*v[i])$

二.

```
#include<iostream>
#include<bits/stdc++.h>
using namespace std;
typedef struct{
    int w;
    int v;
}beg;
int main()
{
    int n,c;
    cin>>n>>c;
    beg a[n+1];
    for(int i=0;i<n;i++)
    {
        cin>>a[i].v>>a[i].w;
    }
    int dp[n+1][c+1];
    memset(dp,0,sizeof(dp));
    for(int i=1;i<=n;i++)
        for(int j=1;j<=c;j++)
        {
            if(j<a[i].w)
                dp[i][j]=dp[i-1][j];
            else if(j<a[i].w*2&& j>=a[i].w)
            {
                dp[i][j]=max(dp[i-1][j],dp[i-1][j-a[i].w]+a[i].v);
            }
            else
            {
                dp[i][j]=max(dp[i-1][j],dp[i-1][j-a[i].w]+a[i].v) ;
                dp[i][j]=max(dp[i][j],dp[i-1][j-2*a[i].w]+2*a[i].v);
            }
        }
    cout<<dp[n][c];
    return 0;
}
```

三.时间复杂度为 $O(n*c)$ ;

第五题:

```
MemoizedLCSLength(X, Y, m, n)
1   for i←0 to m do
2       for j←0 to n do
3           if i=0 or j=0 then
4               c[i, j] ← 0
5           else
6               c[i, j] ← -1
7       LookupLCSLength(X, Y, m, n)
8   return c and b

LookupLCSLength(X, Y, i, j)
1   if c[i, j] ≥ 0 then
2       return c[i, j]
3   if x[i]=y[j] then
4       q← LookupLCSLength(X, Y, i-1, j-1)
5       c[i, j] ← q+1
6       b[i, j] ← “^”
7   else
8       q1← LookupLCSLength(X, Y, i-1, j-1)
9       q2←LookupLCSLength(X, Y, i-1, j-1)
10      if q1 ≤ q2 then
11          c[i, j] ← q1
12          b[i, j] ← “↑”
13      else
14          c[i, j] ← q2
15          b[i, j] ← “←”
```

## 第六题：

把原本借助表b逆序返回LCS改成借助于数组c本身临时判断c[i,j]的值是由c[i-1,j-1], c[i-1,j]和c[i,j-1]中哪一个数值元素所确定  
代价是O(1)时间加上原来重构的时间O (n+m) , 时间还是O (m+n)

```
void printLcs2(int c[maxn][maxn], char x[], char y[],int i,int j) {
    if (i == 0 || j == 0) return;
    if (x[i] == y[j]) {
        printLcs2(c,x,y,i-1,j-1);
        cout<<x[i]<<" ";
    }
    else if (c[i-1][j]>=c[i][j-1]) { printLcs2(c,x,y,i-1,j); }
    else { printLcs2(c,x,y,i,j-1); }
}
```